rqbit UDP 트래커 프로토콜 분석

개요

이 문서는 rqbit BitTorrent 클라이언트의 UDP 트래커 프로토콜 구현을 상세히 분석합니다. UDP 트래커는 HTTP 트래커보다 효율적인 바이너리 프로토콜을 사용하여 더 빠른 피어 검색을 제공합니다.

UDP 트래커 프로토콜 아키텍처

1. 프로토콜 상수 및 기본 타입

액션 타입

```
const ACTION_CONNECT: u32 = 0; // 연결 요청

const ACTION_ANNOUNCE: u32 = 1; // announce 요청

// const ACTION_SCRAPE: u32 = 2; // scrape 요청 (미구현)

const ACTION_ERROR: u32 = 3; // 오류 응답
```

이벤트 타입

```
pub  const EVENT_NONE: u32 = 0; // 일반 announce

pub  const EVENT_COMPLETED: u32 = 1; // 다운로드 완료

pub  const EVENT_STARTED: u32 = 2; // 토렌트 시작

pub  const EVENT_STOPPED: u32 = 3; // 토렌트 중지
```

기본 타입 정의

```
pub type ConnectionId = u64; // 연결 식별자
pub type TransactionId = u32; // 트랜잭션 식별자

const CONNECTION_ID_MAGIC: ConnectionId = 0x41727101980; // 연결 매직 넘버
```

2. 요청 메시지 구조

Request 열거형

```
pub enum Request {
Connect, // 연결 요청
Announce(ConnectionId, AnnounceFields), // announce 요청
}
```

AnnounceFields 구조체

```
pub struct AnnounceFields {
pub info_hash: Id20, // 토렌트 해시
pub peer_id: Id20, // 피어 ID
pub downloaded: u64, // 다운로드된 바이트
pub left: u64, // 남은 바이트
pub uploaded: u64, // 업로드된 바이트
pub event: u32, // 이벤트 타입
```

```
pub key: u32, // 인증 키
pub port: u16, // 리스닝 포트
}
```

3. 메시지 직렬화

Request 직렬화 구현

```
impl Request {
pub fn serialize(
&self,
transaction_id: TransactionId,
buf: &mut [u8],
) -> anyhow::Result<usize> {
struct W<'a> {
buf: &'a mut [u8],
offset: usize,
}
impl W<'_> {
fn extend_from_slice(&mut self, s: &[u8]) -> anyhow::Result<()> {
if self.buf.len() < self.offset + s.len() {
bail!("not enough space in buffer")
}
self.buf[self.offset..self.offset + s.len()].copy_from_slice(s);
```

```
self.offset += s.len();
Ok(())
}
}

let mut w = W { buf, offset: 0 };

match self {
Request::Connect => {
// Connect 요청 형식:
// 8바이트: 연결 매직 넘버
// 4바이트: 액션 (0)
// 4바이트: 트랜잭션 ID
w.extend_from_slice(&CONNECTION_ID_MAGIC.to_be_bytes())?;
w.extend_from_slice(&ACTION_CONNECT.to_be_bytes())?;
w.extend_from_slice(&transaction_id.to_be_bytes())?;
}
Request::Announce(connection_id, fields) => {
// Announce 요청 형식:
// 8바이트: 연결 ID
// 4바이트: 액션 (1)
// 4바이트: 트랜잭션 ID
// 20바이트: info_hash
```

```
// 20바이트: peer_id

// 8바이트: downloaded

// 8바이트: left

// 8바이트: uploaded

// 4바이트: event

// 4바이트: IP 주소 (0 = 기본값)

// 4바이트: key

// 4바이트: num_want (-1 = 기본값)

// 2바이트: port

w.extend_from_slice(&connection_id.to_be_bytes())?;

w.extend_from_slice(&ACTION_ANNOUNCE.to_be_bytes())?;

w.extend_from_slice(&transaction_id.to_be_bytes())?;

w.extend_from_slice(&fields.info_hash.0)?;

w.extend_from_slice(&fields.peer_id.0)?;

w.extend_from_slice(&fields.downloaded.to_be_bytes())?;

w.extend_from_slice(&fields.left.to_be_bytes())?;

w.extend_from_slice(&fields.uploaded.to_be_bytes())?;

w.extend_from_slice(&fields.event.to_be_bytes())?;

w.extend_from_slice(&0u32.to_be_bytes())?; // IP 주소 0

w.extend_from_slice(&fields.key.to_be_bytes())?;

w.extend_from_slice(&(-1i32).to_be_bytes())?; // num_want -1

w.extend_from_slice(&fields.port.to_be_bytes())?;

}
```

```
}
Ok(w.offset)
}
}
```

직렬화 특징:

- 빅 엔디안 바이트 순서 사용
- 고정 길이 필드들로 구성
- 버퍼 오버플로우 방지

4. 응답 메시지 구조

Response 열거형

```
pub enum Response {
Connect(ConnectionId), // 연결 응답
Announce(AnnounceResponse), // announce 응답
Error(String), // 오류 응답
Unknown, // 알 수 없는 응답
}
```

AnnounceResponse 구조체

```
pub struct AnnounceResponse {
pub interval: u32, // announce 간격 (초)
```

```
pub leechers: u32, // 리처 수
pub seeders: u32, // 시더 수
pub addrs: Vec<SocketAddr>, // 피어 주소 목록
}
```

5. 응답 파싱

숫자 파싱 트레이트

```
trait ParseNum: Sized {
fn parse_num(buf: &[u8]) -> anyhow::Result<(Self, &[u8])>;
}

// 매크로를 통한 구현 생성
macro_rules! parse_impl {
($ty:tt, $size:expr) => {
impl ParseNum for $ty {
fn parse_num(buf: &[u8]) -> anyhow::Result<($ty, &[u8])> {
let (bytes, rest) = split_slice(buf, $size)
.with_context(|| format!("expected {} bytes", $size))?;
let num = $ty::from_be_bytes(s_to_arr(bytes));
Ok((num, rest))
}
}
```

```
};
}

// 다양한 정수 타입에 대한 구현
parse_impl!(u32, 4);
parse_impl!(u64, 8);
parse_impl!(u16, 2);
parse_impl!(i32, 4);
// ... 기타 타입들
```

Response 파싱 구현

```
impl Response {
pub fn parse(buf: &[u8], is_ipv6: bool) -> anyhow::Result<(TransactionId, Self)>
let (action, buf) = u32::parse_num(buf).context("can't parse action")?;
let (tid, buf) = u32::parse_num(buf).context("can't parse transaction id")?;

let response = match Self::parse_response(action, is_ipv6, buf) {
Ok(r) => r,
Err(e) => {
debug!("error parsing: {e:#}");
Response::Unknown
}
};
```

```
Ok((tid, response))

}


fn parse_response(action: u32, is_ipv6: bool, mut buf: &[u8]) -> anyhow::Result<

let response = match action {

ACTION_CONNECT => {

// Connect 응답: 8바이트 연결 ID

let (connection_id, b) = u64::parse_num(buf)

.context("can't parse connection id")?;

buf = b;

Response::Connect(connection_id)

}

ACTION_ANNOUNCE => {

// Announce 응답 파싱

let (interval, b) = u32::parse_num(buf).context("can't parse interval")?;

let (leechers, b) = u32::parse_num(b).context("can't parse leechers")?;

let (seeders, mut b) = u32::parse_num(b).context("can't parse seeders")?;

// 피어 주소 목록 파싱

let mut addrs = Vec::new();

while !b.is_empty() {

let (addr, b2) = if is_ipv6 {

// IPv6: 16바이트 IP + 2바이트 포트
```

```
let (ip, b2) = u128::parse_num(b)?;
(IpAddr::V6(Ipv6Addr::from(ip)), b2)
} else {
// IPv4: 4바이트 IP + 2바이트 포트
let (ip, b2) = u32::parse_num(b)?;
(IpAddr::V4(Ipv4Addr::from(ip)), b2)
};
b = b2;


let (port, b2) = u16::parse_num(b)?;
b = b2;
addrs.push(SocketAddr::new(addr, port));
}
buf = b;
Response::Announce(AnnounceResponse {
interval,
leechers,
seeders,
addrs,
})
}
ACTION_ERROR => {
// 오류 응답: null-terminated 문자열
```

```
let msg = CStr::from_bytes_with_nul(buf)
.ok()
.and_then(|s| s.to_str().ok())
.or_else(|| std::str::from_utf8(buf).ok())
.unwrap_or("<invalid UTF-8>")
.to_owned();
return Ok(Response::Error(msg));
}
_ => bail!("unsupported action {action}"),
};


// 남은 바이트 검증
if !buf.is_empty() {
bail!(
"parsed {response:?} so far, but got {} remaining bytes",
buf.len()
);
}


Ok(response)
}
}
```

6. UDP 클라이언트 구현

UdpTrackerClient 구조체

```
#[derive(Clone)]
pub struct UdpTrackerClient {
state: Arc<ClientShared>,
}

struct ClientShared {
sock: UdpSocket, // UDP 소켓
locked: RwLock<ClientLocked>, // 동기화된 상태
}

struct ClientLocked {
connections: HashMap<SocketAddr, ConnectionIdMeta>, // 연결 ID 캐시
transactions: HashMap<TransactionId, tokio::sync::oneshot::Sender<Response>>, //
}
```

연결 ID 관리

```
struct ConnectionIdMeta {
id: ConnectionId,
created: Instant,
```

```
}

async fn get_connection_id(&self, addr: SocketAddr) -> anyhow::Result<Connection
// 캐시된 연결 ID 확인 (60초 유효)
if let Some(m) = self.state.locked.read().connections.get(&addr) {
if m.created.elapsed() < Duration::from_secs(60) {
return Ok(m.id);
}
}

// 새 연결 ID 요청
let response = self.request(addr, Request::Connect).await?;
match response {
Response::Connect(connection_id) => {
self.state.locked.write().connections.insert(
addr,
ConnectionIdMeta {
id: connection_id,
created: Instant::now(),
},
);
Ok(connection_id)
}
```

```
_ => anyhow::bail!("expected connect response"),
}
}
```

7. 트랜잭션 추적 시스템

트랜잭션 ID 관리

```
fn reserve_transaction_id(
&self,
tx: tokio::sync::oneshot::Sender<Response>,
) -> anyhow::Result<TransactionIdGuard<'_>> {
let mut g = self.state.locked.write();
// 최대 10번 시도하여 고유한 트랜잭션 ID 생성
for _ in 0..10 {
let t = new_transaction_id();
match g.transactions.entry(t) {
Entry::Occupied(_) => continue,
Entry::Vacant(vac) => {
vac.insert(tx);
return Ok(TransactionIdGuard {
tid: t,
state: &self.state,
});
```

```
}
}
}
bail!("cant generate transaction id")
}

pub fn new_transaction_id() -> TransactionId {
rand::rng().random()
}
```

TransactionIdGuard RAII 패턴

```
struct TransactionIdGuard<'a> {
tid: TransactionId,
state: &'a ClientShared,
}

impl Drop for TransactionIdGuard<'_> {
fn drop(&mut self) {
// 트랜잭션 완료 시 자동 정리
let mut g = self.state.locked.write();
g.transactions.remove(&self.tid);
}
```

```
}
```

8. 비동기 요청/응답 처리

요청 전송 및 응답 대기

```
async fn request(&self, addr: SocketAddr, request: Request) -> anyhow::Result<Re

// 1. 트랜잭션 ID 예약

let (tx, rx) = tokio::sync::oneshot::channel();

let tid_g = self.reserve_transaction_id(tx)?;



// 2. 요청 직렬화 및 전송

let mut write_buf = [0u8; 1024];

let len = request.serialize(tid_g.tid, &mut write_buf)?;

self.state

.sock

.send_to(&write_buf[..len], addr)

.await

.with_context(|| format!("error sending to {addr:?}"))?;



// 3. 응답 대기 (10초 타임아웃)

let response = tokio::time::timeout(Duration::from_secs(10), rx)

.await

.context("timeout connecting")?
```

```
.context("sender dead")?;

// 4. 오류 응답 처리

match &response {

Response::Error(e) => {

anyhow::bail!("remote errored: {e}")

}

Response::Unknown => {

anyhow::bail!("remote replied with something we could not parse")

}

_ => {}

}

Ok(response)

}
```

응답 수신 루프

```
async fn run(self) -> anyhow::Result<()> {

let mut buf = [0u8; 16384];

loop {

// 1. UDP 패킷 수신

let (len, addr) = match self.state.sock.recv_from(&mut buf).await {

Ok(r) => r,

Err(e) => {

warn!("error in UdpSocket::recv_from: {e:#}");
```

```
continue;
}
};

// 2. 응답 파싱
let (tid, response) = match Response::parse(&buf[..len], addr.is_ipv6()) {
Ok(r) => r,
Err(e) => {
debug!(?addr, "error parsing UDP response: {e:#}");
continue;
}
};

trace!(?tid, ?response, ?addr, "received");

// 3. 해당 트랜잭션에 응답 전달
let t = self.state.locked.write().transactions.remove(&tid);
match t {
Some(tx) => match tx.send(response) {
Ok(_) => {}
Err(_) => {
debug!(tid, "reader dead");
}
```

```
},
None => {
debug!(tid, "nowhere to send response");
}
};
}
}
```

9. UDP 패킷 손실 처리

재전송 로직

UDP는 신뢰성이 없는 프로토콜이므로 상위 계층에서 재전송을 처리합니다:

```
// TrackerComms에서의 재시도 로직
match self.tracker_one_request_udp(addr, &client).await {
Ok(sleep) => sleep_interval = Some(sleep),
Err(_) => {
// 오류 시 기본 간격으로 재시도
sleep_interval = Some(sleep_interval.unwrap_or(Duration::from_secs(60)))
}
}
```

타임아웃 처리

```
let response = tokio::time::timeout(Duration::from_secs(10), rx)
.await
.context("timeout connecting")?;
```

10. IPv4/IPv6 듀얼스택 지원

주소 해석

```
async fn udp_tracker_to_socket_addrs(
host: url::Host<&str>,
port: u16,
) -> anyhow::Result<UdpTrackerResolveResult> {
let res = match host {
url::Host::Domain(name) => {
let mut v4: Option<SocketAddrV4> = None;
let mut v6: Option<SocketAddrV6> = None;
// IPv4와 IPv6 주소를 각각 하나씩 수집
for addr in tokio::net::lookup_host((name, port)).await? {
match (v4, v6, addr) {
(None, _, SocketAddr::V4(addr)) => v4 = Some(addr),
(_, None, SocketAddr::V6(addr)) => v6 = Some(addr),
_ => continue,
}
}
```

```
match (v4, v6) {
(Some(v4), Some(v6)) => UdpTrackerResolveResult::Two(v4, v6),
(Some(v4), None) => UdpTrackerResolveResult::One(v4.into()),
(None, Some(v6)) => UdpTrackerResolveResult::One(v6.into()),
_ => anyhow::bail!("zero addresses returned looking up {name}"),
}
}
url::Host::Ipv4(addr) => UdpTrackerResolveResult::One((addr, port).into()),
url::Host::Ipv6(addr) => UdpTrackerResolveResult::One((addr, port).into()),
};
Ok(res)
}
```

병렬 요청 처리

```
UdpTrackerResolveResult::Two(v4, v6) => {
let (r4, r6) = tokio::join!(
self.tracker_one_request_udp(v4.into(), &client),
self.tracker_one_request_udp(v6.into(), &client)
);
sleep_interval = Some(
r4.or(r6)
.ok()
```

```
.or(sleep_interval)

.unwrap_or(Duration::from_secs(60)),

)

}
```

11. 성능 최적화 기법

1. 연결 ID 캐싱

- 60초 동안 연결 ID 재사용
- 불필요한 connect 요청 최소화

2. 효율적인 버퍼 관리

- 고정 크기 버퍼 사용 (16KB)
- 스택 할당을 통한 메모리 효율성

3. 비동기 처리

- 논블로킹 UDP 소켓
- 동시 다중 트래커 지원

4. 트랜잭션 추적 최적화

- HashMap을 통한 O(1) 트랜잭션 조회
- RAII 패턴을 통한 자동 정리

12. 보안 고려사항

1. 트랜잭션 ID 보안

- 암호학적으로 안전한 난수 생성

- 충돌 방지를 위한 재시도 로직

2. 버퍼 오버플로우 방지

- 엄격한 버퍼 크기 검사
- 파싱 시 길이 검증

3. 타임아웃 보호

- 10초 요청 타임아웃
- 무한 대기 방지

결론

rqbit의 UDP 트래커 프로토콜 구현은 다음과 같은 특징을 가집니다:

1. **효율적인 바이너리 프로토콜**: HTTP보다 빠른 통신
2. **견고한 오류 처리**: 패킷 손실과 타임아웃에 대한 적절한 대응
3. **듀얼스택 지원**: IPv4와 IPv6 동시 지원
4. **성능 최적화**: 연결 ID 캐싱과 비동기 처리
5. **보안 고려**: 안전한 트랜잭션 ID 생성과 버퍼 보호

이러한 구현을 통해 UDP 트래커와의 안정적이고 효율적인 통신을 보장합니다.